

RADAR B3 PRO IA — Verificação da v3.0 · Espec. v3.1

Projeto: TESTE_BRAPI **Base verificada:** commit [308f47a](#) **Data:** 31/08/2026 **Método:** suíte do projeto (94 testes) + 12 testes adversariais independentes sobre a cadeia real do ITUB4

0. Placar

8 dos 11 itens corrigidos e verificados. 3 em aberto, sendo um deles a repetição do padrão que a nota de método da v3.0 pedia para evitar.

#	Item	Status	Evidência medida
01	G4 — lastTradeDate no mapper	✓	série sem o campo → inelegível ; com data antiga → inelegível
02	G5 — fixture real do ITUB4	✓	8 calls reais: ITUBI407@40,05 ... ITUBI424@41,80
03	G3 — gate de IV	⚠ meio-feito	squeeze bloqueia; IV_INDISPONIVEL neutralizado por fallback
04	G2 — diagnóstico verdadeiro	✓	"2 séries · 2 elegíveis · 0 pares" + a restrição que eliminou
05	G1 — Iron Condor	✓	80 condors válidos , deltas -0,257 / +0,266, payoff máx no centro

#	Item	Status	Evidência medida
06	G6 — UI trata EM_ANALISE	✗ não feito	e o padrão foi replicado na página nova
07	N105 — afordâncias	⚠ parcial	28 testids, 0 onClick em div; faltam 5 testids e 3 atributos ARIA
08	N102 — /api/options	✓	devolve electedOptionStr ategy, operation, fundamentals
09	N101 — painel na aba de opções	✓	options-eligible -structures-pane l + payoff renderizados
10	N103 — alternativa a débito	✗ meio-feito	contrato e UI prontos; motor nunca preenche
11	N104 — visões do Panorama	✓	3 abas com role="tab" e aria-selected

Suíte do projeto: **94 testes, 94 verdes**. Meus testes adversariais: **11 de 12** — o que falhou é o item 03.

1. O destaque positivo — Iron Condor

O item de maior severidade foi resolvido bem. Antes, o cenário 3 produzia bloqueio com asas de R\$ 0,09 e perna vendida em delta -0,418. Agora, sobre a cadeia real:

Iron Condor a Crédito — faixa R\$ 37,96 a R\$ 41,05

COMPRA ITUBU379 K 37,21 δ -0,171 @ 0,24

VENDA ITUBU385 K 37,96 δ -0,257 @ 0,40

VENDA ITUBI417 K 41,05 δ +0,266 @ 0,37

COMPRA ITUBI424 K 41,80 δ +0,182 @ 0,23

diagnóstico: 26 séries na cadeia · 23 elegíveis · 80 condors válidos

payoff: extremo inferior -0,45 · centro +0,30 · extremo superior -0,45

As duas pernas vendidas estão em 0,257 e 0,266 — praticamente simétricas, dentro do teto de 0,30. As asas ficaram em 0,75 de cada lado, simetria perfeita. O payoff tem máximo no centro, como deve ser. É uma estrutura que um operador montaria.

O diagnóstico do item 04 também ficou bom. Mensagem completa quando bloqueia:

"Há séries negociáveis, mas nenhuma combinação atende à largura e ao prêmio mínimos da trava. **Melhor combinação encontrada: largura de 0,63% do spot, abaixo do mínimo de 1,5%.**"

É exatamente o que a spec pedia: o número real e a restrição que eliminou os pares.

2. Item 03 — o gate de IV tem um fallback que o anula

Severidade:  crítica · **Arquivo:** [cme-election.ts:450-482](#)

Evidência

Teste: optionAnalysis.ivAtm = null, resto igual

Resultado: status AUTORIZADA · reason: undefined

O gate foi escrito, mas antes de disparar ele **reconstrói a IV por conta própria**:

```
let atmlv = optionAnalysis?.ivAtm?.calllv;
```

```
if (!atmlv || atmlv <= 0) {
```

```
  const validlvSeries = analyticsList.filter((a) => (a.impliedVolatility || 0) > 0);
```

```
  if (validlvSeries.length > 0) {
```

```

const closestToSpot = validIvSeries.reduce((prev, curr) =>

  Math.abs(curr.strike - spot) < Math.abs(prev.strike - spot) ? curr : prev);

atmlv = closestToSpot.impliedVolatility; // ← anula o gate

}

}

if (!atmlv || atmlv <= 0) { /* IV_INDISPONIVEL — praticamente inalcançável */ }

```

Três problemas nesse fallback:

1. **Ele contorna todo o controle de qualidade da v2.2.** O `ivAtm` que vem de `options-barriers` é mediana de séries elegíveis dentro de $\pm 5\%$ do spot, com OI mínimo, descarte de outlier e verificação de divergência call/put. O fallback pega **uma única série**, a mais próxima do spot, **sem nenhum desses critérios** — pode ser uma série de R\$ 0,03, com 200 contratos, IV de 90%.
2. **IV_INDISPONIVEL virou código morto.** Ele só dispara se nenhuma série da cadeia inteira tiver IV — situação que praticamente não ocorre. A RN-522 continua não valendo.
3. **É a mesma classe de defeito que a v2.1 eliminou nos prêmios.** O `|| width * 0.40` foi removido porque fabricava número; este fabrica volatilidade. Fallback silencioso que transforma ausência de dado em aprovação.

Correção

Remover o bloco de reconstrução por inteiro. O gate fica:

```

const atmlv = optionAnalysis?.ivAtm?.callIv;

if (!atmlv || atmlv <= 0) {

  return createBlockedStrategy(..., 'IV_INDISPONIVEL', diagnostics, priceContext);

}

```

Se a IV ATM confiável não existe, quem decide isso é `options-barriers`, com os critérios da v2.2. O motor de eleição **consome** essa decisão; não a refaz com régua mais frouxa.

Observação de escopo: o gate hoje é aplicado quando `isCreditTarget` é verdadeiro, e essa flag cobre ALTA, BAIXA e LATERAL — ou seja, tudo. Estruturas **a débito** não vendem volatilidade e não deveriam ser bloqueadas por ausência de IV; devem seguir com o aviso "regime de volatilidade não confirmado" (CT-960). Separar as duas condições ao remover o fallback.

Teste

ID	Cenário	Esperado
CT-958	<code>ivAtm = null</code> , alvo a crédito	<code>IV_INDISPONIVEL</code>
CT-958b	<code>ivAtm = null</code> , cadeia com IVs válidas	ainda <code>IV_INDISPONIVEL</code> — nenhuma reconstrução
CT-960	<code>ivAtm = null</code> , alvo a débito	eleita, com aviso de regime não confirmado

3. Item 06 — não implementado, e o padrão foi replicado

Severidade: 🟡 alta · **Arquivos:** `QuoteView.tsx:916,953` e `OptionsBarriersView.tsx:530,604`

`isActionable` não existe no projeto. A condição continua a mesma dos dois lados:

// `QuoteView.tsx:953`

```
{!isBlocked && execMode === 'OPTIONS' && elected && elected.status !== 'BLOQUEADA' && elected.legs.length > 0 && (...)}
```

// `OptionsBarriersView.tsx:530` ← página nova, mesmo padrão

```
{data.electedOptionStrategy && data.electedOptionStrategy.status !== 'BLOQUEADA' && data.electedOptionStrategy.legs.length > 0 && (...)}
```

Hoje o defeito é latente — confirmei que o bloqueio por drift devolve `EM_ANALISE` com `legs.length === 0`, então cai no painel certo por acidente de implementação, não por regra. Basta um caminho futuro devolver `EM_ANALISE` com pernas (uma estrutura válida com advertência de defasagem, que é justamente o comportamento previsto na v2.1 para drift entre 1,5% e 3%) para ela ser renderizada como autorizada, sem tarja.

O ponto que me preocupa mais: **a página nova nasceu com o defeito**. A v3.0 tinha o item 06 listado e a nota de método pedindo para procurar o padrão no projeto inteiro; a implementação do item 09 copiou a condição defeituosa em vez da corrigida.

Correção

Uma constante compartilhada, usada nos dois componentes:

```
// lib/domain/cme-election.ts (ou util compartilhado)
```

```
export const isActionableStrategy = (s?: ElectedOptionStrategy | null) =>
```

```
  s?.status === 'AUTORIZADA' && s.legs.length > 0;
```

EM_ANALISE ganha painel próprio: estrutura visível **com** tarja de advertência e **sem** bloco de envio de ordem. Zero comparações de **status** espalhadas pelos componentes.

Teste

ID	Cenário	Esperado
CT-967	EM_ANALISE com pernas, nos dois componentes	painel de advertência, sem bloco de ordem
CT-967c	varredura estática	nenhuma comparação direta a ' BLOQUEADA ' fora do util

4. Item 10 — a alternativa nunca é produzida

Severidade: 🟡 alta · **Arquivo:** **cme-election.ts**

O tipo tem o campo (**alternative?** na linha 63) e a UI tem o controle com **disabled={!data.electedOptionStrategy.alternative}**. Mas **não existe nenhuma atribuição a **alternative** em todo o motor** — **grep** por **alternative**: retorna zero ocorrências de escrita.

Teste: COMPRA no ITUB4, cadeia real

eleita: Trava de Alta com Put a Crédito (Bull Put 38,46 / 37,21)

alternativa: AUSENTE

Resultado prático: o botão "Alternativa" está **permanentemente desabilitado**. O usuário vê um controle morto — o que é pior que não ter controle, porque sugere uma funcionalidade que não existe.

Correção

Após eleger a estrutura principal, rodar o caminho oposto (débito quando a eleita for a crédito, e vice-versa) com **as mesmas validações** — elegibilidade, faixa de delta, largura, crédito/largura, `assertDirection`. Se passar, preencher:

alternative: {

strategy: alternativeStructure,

rationale: `Estrutura a débito não foi eleita porque a IV ATM de \${ivAtm}% indica regime \${regime.label}, que favorece venda de prêmio.`,

}

Se não passar, `alternative` fica `undefined` — e aí o botão desabilitado é honesto. Nesse caso, a UI deve exibir o motivo no tooltip (CT-N06), o que hoje também não acontece porque não há motivo para exibir.

5. Item 07 — parcial

O grosso foi feito e bem: **28 data-testid**, **zero onClick** em `<div>` ou `<span>`, `role="tab"` em 12 controles e `role="tablist"` em 5 contêineres. Isso resolve as três falhas de automação que originaram a Fase B.

Faltam quatro coisas, todas pequenas:

Faltante	Impacto
<code>nav-tab-overview</code> , <code>nav-tab-quote</code> , <code>nav-tab-screener</code> , <code>nav-tab-options</code> , <code>nav-tab-help</code>	a navegação principal — justamente o ponto de entrada de qualquer teste E2E — é o único bloco sem testid
<code>role="tabpanel"</code> (0 ocorrências)	os painéis não se declaram como conteúdo das abas

Faltante	Impacto
<code>aria-controls</code> (0 ocorrências)	não há ligação entre aba e painel
<code>aria-live="polite"</code> (0 ocorrências)	veredito, estrutura e plano de trade mudam após carregamento sem anunciar

Nada disso quebra funcionalidade; é conclusão do item.

6. Decisão ainda pendente — pesos do score

Continua valendo o registrado na seção 14 da v3.0, e não foi decidida. A perna vendida da trava vertical no ITUB4 permanece em **δ -0,328** (alvo 0,28, teto 0,38), com a série de δ -0,295 disponível na cadeia.

	Atual	Proposta
delta da vendida	0,40	0,50
delta da comprada	0,15	0,15
retorno sobre risco	0,25	0,20
liquidez efetiva	0,20	0,15
POP estimada (ITUB4)	~67%	~71%

Vale notar que o Iron Condor **já saiu bem calibrado** com os pesos novos que a v3.0 definiu para ele (0,35 delta / 0,25 retorno / 0,20 simetria / 0,20 liquidez) — as vendas ficaram em 0,257 e 0,266 contra alvo 0,20. Isso reforça a proposta: onde o delta pesa mais, o resultado fica mais aderente ao alvo.

7. Fila de correção

- Item 03 remover o fallback de IV  ~15 min
- Item 10 produzir a alternativa  ~1 h
- Item 06 isActionable nos dois componentes  ~20 min

4. Item 07 5 testids + tabpanel/controls/live 🟡 ~30 min

5. Decisão dos pesos do score — sua definição

O item 03 primeiro, e por um motivo específico: enquanto o fallback existir, **qualquer teste de regime de volatilidade dá falso verde**. Ele mascara o próprio gate que deveria validar.

8. Critérios de aceite desta rodada

1. `ivAtm = null` bloqueia estrutura a crédito **mesmo com a cadeia cheia de IVs válidas**.
 2. Estrutura a débito não é bloqueada por ausência de IV ATM; exibe aviso de regime não confirmado.
 3. Nenhuma reconstrução de IV dentro de `cme-election.ts` — a IV ATM vem exclusivamente de `options-barriers`.
 4. `alternative` preenchido sempre que a estrutura oposta passar em todas as validações; `undefined` com motivo exibido quando não passar.
 5. `isActionableStrategy` é a única forma de decidir se uma estrutura é acionável, nos dois componentes.
 6. Os cinco `nav-tab-*` presentes; `role="tabpanel"`, `aria-controls` e `aria-live` aplicados.
 7. Suíte verde, incluindo CT-958b, CT-960, CT-967c e CT-N06.
-

9. Nota de método — a terceira ocorrência

O item 06 é a terceira vez nesta série de correções que o mesmo padrão aparece: **a correção é aplicada onde o defeito foi relatado, não onde o padrão existe**.

- v2.2: seleção por índice corrigida nas travas verticais, mantida no Iron Condor.
- v3.0 item 01: filtro novo passando no teste porque a fixture tinha um campo que produção não preenchia.
- v3.1 item 06: condição defeituosa **copiada** para o componente novo, no mesmo commit que deveria corrigi-la.

Os três foram encontrados por teste adversarial escrito de fora, não pela suíte do projeto — e a suíte estava verde nas três ocasiões, com 87, 87 e 94 testes.

Isso não é crítica ao volume de testes; é sobre o tipo. Uma suíte construída junto com a implementação testa o que o autor pensou. O que pega esses casos é a pergunta feita de fora: *"onde mais esse padrão existe?"* e *"esse teste passaria se o dado viesse como a fonte manda?"*

Sugestão concreta, e barata: antes de fechar cada um dos quatro itens da fila, rodar `grep` do padrão corrigido no projeto inteiro e colar o resultado no PR. Nos três casos acima, um `grep` de dez segundos — `sorted[0], lastTradeDate, !== 'BLOQUEADA'` — teria evitado a rodada seguinte.

Anexo — limpeza

Os `.tgz` anteriores saíram do repositório e o `.gitignore` foi ajustado — obrigado. Deixei um `_qa3.tgz` novo na raiz nesta rodada de verificação, pelo mesmo motivo de sempre (o `node_modules` local tem binário Windows do rollup e não roda na VM Linux). Se o `.gitignore` já cobre `*.tgz`, ele não entra em commit; pode apagar quando quiser.